Universidad de Guadalajara

Centro Universitario de Ciencias Exactas e Ingenierías

Ingeniería informática



Seguridad defensiva y gestión

Actividad: Proyecto Final - Cinemática Inversa Robot 5 GDL

Ángel Jean Fernández Argoul

1. MARCO TEÓRICO

El problema de la **Cinemática Inversa (IK)** en robótica consiste en determinar los valores de las articulaciones de un manipulador ($q_1, q_2, \ldots, q_n$) necesarios para que su efector final alcance una posición y orientación deseadas en el espacio cartesiano. Debido a que las ecuaciones resultantes son altamente no lineales (compuestas por multiplicaciones de senos y cosenos), a menudo existen múltiples soluciones o singularidades, lo que hace que los enfoques analíticos clásicos sean complejos.

Para resolver esto, utilizamos **Particle Swarm Optimization (PSO)**. Esta metaheurística modela el problema como un enjambre de partículas donde cada una representa una configuración articular candidata. Las partículas "vuelan" por el espacio de búsqueda ajustando sus ángulos (para articulaciones de revolución) y su desplazamiento (para articulaciones prismáticas) guiándose por su mejor recuerdo personal y el mejor recuerdo del enjambre hasta minimizar el error de posición espacial.

2. DESARROLLO Y CÓDIGO COMENTADO

El algoritmo fue implementado en MATLAB, tomando como base las especificaciones de Denavit-Hartenberg del manipulador R-R-P-R-R.

2.1 Matriz de Transformación Homogénea

Para calcular la cinemática directa, se utilizó la matriz de transformación entre eslabones dictada en las instrucciones del proyecto :

$$T = \begin{bmatrix} \cos(\theta_i) & -\sin(\theta_i)\cos(\alpha_i) & \sin(\theta_i)\sin(\alpha_i) & b_i\cos(\theta_i) \\ \sin(\theta_i) & \cos(\theta_i)\cos(\alpha_i) & -\cos(\theta_i)\sin(\alpha_i) & b_i\sin(\theta_i) \\ 0 & \sin(\alpha_i) & \cos(\alpha_i) & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

2.2 Función de Aptitud (Fitness) y Puntos Extra

La función objetivo base evalúa el error euclidiano e entre la posición actual y la deseada, maximizando el fitness a través de la fórmula $f(q) = \frac{1}{1+e}$.

Para lograr los puntos extra, la función de costo se modificó para incluir **penalizaciones**:

1. **Continuidad Articular :** Se añadió el término $\Delta q = (q_{old} - q_{new})$ a la función de error . Al buscar la segunda trayectoria, el algoritmo penaliza los cambios bruscos de configuración, obligando a las articulaciones a buscar la solución más cercana a la postura actual.
2. **Orientación con Cuaterniones :** La matriz de rotación extraída de T_{05} se convierte en un cuaternión unitario . El error de orientación se calcula mediante el producto punto entre el cuaternión deseado y el actual, combinándose de forma ponderada con el error de posición.

3.3 Código MATLAB Completo

```
% Proyecto Final - Cinematica Inversa con PSO (Robot 5 GDL R-R-P-R-R)
clc; clear; close all;

%% PARAMETROS DEL PSO
num_particulas = 50;
iteraciones_max = 200;
c1 = 2.05; c2 = 2.05; % Constantes cognitiva y social
w_max = 0.9; w_min = 0.4; % Inercia dinamica

% Limites de las articulaciones [q1, q2, q3, q4, q5]
% q1, q2, q4, q5 rotacionales [-pi, pi]. q3 prismatica [0.1, 1.0] metros.
lim_inf = [-pi, -pi, 0.1, -pi, -pi];
lim_sup = [ pi,  pi, 1.0,  pi,  pi];
dim = 5;

%% TAREA 1: Alcanzar el Punto Objetivo 1 (Pose completa)
% Posicion deseada
Pos_deseada = [0.4, 0.3, 0.5];
% Orientacion deseada en Cuaterniones [w, x, y, z] (Punto Extra 2)
Quat_deseado = [1, 0, 0, 0]; % Orientacion sin rotacion relativa

% Ejecutar PSO para el Punto 1 sin penalizacion de continuidad
fprintf('Optimizando para el Punto 1...\n');
q_inicial = zeros(1, 5); % No hay configuracion previa
[gbest_q1, gbest_fit1, historico_fit1] = ejecutarPSO(num_particulas,
iteraciones_max, dim, lim_inf, lim_sup, c1, c2, w_max, w_min, Pos_deseada,
Quat_deseado, q_inicial, false);

%% TAREA 2: Continuidad Articular (Punto Extra 1)
% Definimos un Punto 2 muy cercano espacialmente
Pos_deseada2 = [0.45, 0.35, 0.5];
```

```
fprintf('Optimizando para el Punto 2 (Garantizando continuidad
articular)...\n');
% Pasamos gbest_q1 como postura anterior y activamos la penalizacion
[gbest_q2, gbest_fit2, historico_fit2] = ejecutarPSO(num_particulas,
iteraciones_max, dim, lim_inf, lim_sup, c1, c2, w_max, w_min, Pos_deseada2,
Quat_deseado, gbest_q1, true);

%% GRAFICACION DE RESULTADOS

% 1. Grafica de Convergencia
figure('Name', 'Convergencia del Fitness');
plot(1:iteraciones_max, historico_fit1, 'b', 'LineWidth', 2); hold on;
plot(1:iteraciones_max, historico_fit2, 'r', 'LineWidth', 2);
title('Gráfica de Convergencia del PSO');
xlabel('Generación'); ylabel('Fitness ( 1 / (1+e) )');
legend('Trayectoria a Punto 1', 'Trayectoria a Punto 2 (Continuidad)');
grid on;

% 2. Grafica 3D del Robot (Postura Final Punto 2)
[~, Puntos3D] = forwardKinematics(gbest_q2);
figure('Name', 'Cinematica Directa - Robot 3D');
plot3(Puntos3D(:,1), Puntos3D(:,2), Puntos3D(:,3), '-ko', 'LineWidth', 3,
'MarkerSize', 8, 'MarkerFaceColor', 'b');
hold on; grid on; axis equal;
% Marcar el end-effector y los objetivos
plot3(Puntos3D(end,1), Puntos3D(end,2), Puntos3D(end,3), 'xb', 'MarkerSize',
15, 'LineWidth', 3);
plot3(Pos_deseada(1), Pos_deseada(2), Pos_deseada(3), '*r', 'MarkerSize',
10);
plot3(Pos_deseada2(1), Pos_deseada2(2), Pos_deseada2(3), '*g', 'MarkerSize',
10);
xlabel('X (m)'); ylabel('Y (m)'); zlabel('Z (m)');
title('Postura final del robot de 5 GDL');
legend('Eslabones', 'Efector Final', 'Objetivo 1', 'Objetivo 2 (Actual)');
view(45, 30);

%% --- FUNCIONES AUXILIARES ---

function [gbest_pos, gbest_val, historico_fit] = ejecutarPSO(N, iterMax,
dim, L_inf, L_sup, c1, c2, wmax, wmin, P_des, Q_des, q_old,
aplicar_continuidad)
    % Inicializacion
    pos = zeros(N, dim);
    vel = zeros(N, dim);
    for i = 1:dim
        pos(:, i) = L_inf(i) + rand(N, 1) * (L_sup(i) - L_inf(i));
```

```
        vel(:, i) = -0.1 + rand(N, 1) * 0.2;
    end

    pbest_pos = pos;
    pbest_val = zeros(N, 1);
    gbest_val = -inf;
    gbest_pos = zeros(1, dim);
    historico_fit = zeros(1, iterMax);

    for i = 1:N
        pbest_val(i) = calcularFitness(pos(i,:), P_des, Q_des, q_old,
aplicar_continuidad);
        if pbest_val(i) > gbest_val
            gbest_val = pbest_val(i);
            gbest_pos = pos(i,:);
        end
    end

    % Bucle principal PSO
    for t = 1:iterMax
        w = wmax - (wmax - wmin) * (t / iterMax); % Inercia dinamica
        for i = 1:N
            r1 = rand(1, dim);
            r2 = rand(1, dim);

            % Ecuacion de velocidad
            vel(i,:) = w*vel(i,:) + c1*r1.*(pbest_pos(i,:) - pos(i,:)) +
c2*r2.*(gbest_pos - pos(i,:));
            % Ecuacion de posicion
            pos(i,:) = pos(i,:) + vel(i,:);

            % Restriccion de limites
            pos(i,:) = max(pos(i,:), L_inf);
            pos(i,:) = min(pos(i,:), L_sup);

            % Evaluacion
            fit_actual = calcularFitness(pos(i,:), P_des, Q_des, q_old,
aplicar_continuidad);
            if fit_actual > pbest_val(i)
                pbest_val(i) = fit_actual;
                pbest_pos(i,:) = pos(i,:);
            end
            if fit_actual > gbest_val
                gbest_val = fit_actual;
                gbest_pos = pos(i,:);
            end
```

```
        end
        historico_fit(t) = gbest_val;
    end
end

function fit = calcularFitness(q, P_des, Q_des, q_old, aplicar_continuidad)
    [T05, ~] = forwardKinematics(q);

    % 1. Error Euclidiano (Posicion)
    Pos_actual = T05(1:3, 4)';
    e_pos = norm(P_des - Pos_actual);

    % 2. Error de Orientacion con Cuaterniones (Punto Extra 2)
    R = T05(1:3, 1:3);
    Q_actual = rotm2quat_custom(R); % Funcion personalizada abajo
    e_ori = 1 - abs(dot(Q_des, Q_actual)); % Producto punto para similitud
de cuaterniones

    % 3. Penalizacion de Continuidad (Punto Extra 1)
    e_cont = 0;
    if aplicar_continuidad
        e_cont = sum((q - q_old).^2); % Minimizar el delta_q
    end

    % Funcion objetivo combinada
    % Ponderamos: Posicion importa mas, luego orientacion, luego continuidad
    Error_Total = e_pos + (0.5 * e_ori) + (0.1 * e_cont);

    % f(q) = 1 / (1 + e) -> Maximizamos esto
    fit = 1 / (1 + Error_Total);
end

function [T05, Puntos] = forwardKinematics(q)
    % Parametros D-H [theta, alpha, b, d]
    % q(3) es prismatica (d3). Las demas son rotacionales (theta)
    T01 = matrizDH(q(1), 0.5,  0, 0);
    T12 = matrizDH(q(2), 0.4,  0, 0);
    T23 = matrizDH(0,    0,    0, q(3));
    T34 = matrizDH(q(4), 2.02, 0, 0);
    T45 = matrizDH(q(5), 0.3,  0, 0);

    T02 = T01 * T12;
    T03 = T02 * T23;
    T04 = T03 * T34;
    T05 = T04 * T45; % Matriz final del efector
```

```
    % Extraer coordenadas espaciales para la grafica 3D
    Puntos = zeros(6, 3);
    Puntos(1, :) = [0, 0, 0]; % Origen
    Puntos(2, :) = T01(1:3, 4)';
    Puntos(3, :) = T02(1:3, 4)';
    Puntos(4, :) = T03(1:3, 4)';
    Puntos(5, :) = T04(1:3, 4)';
    Puntos(6, :) = T05(1:3, 4)';
end

function T = matrizDH(theta, alpha, b, d)
    % Matriz de transformacion general reportada en el proyecto
    T = [cos(theta), -sin(theta)*cos(alpha),  sin(theta)*sin(alpha),
b*cos(theta);
         sin(theta),  cos(theta)*cos(alpha), -cos(theta)*sin(alpha),
b*sin(theta);
         0,           sin(alpha),             cos(alpha),            d;
         0,           0,                      0,                     1];
end

function q = rotm2quat_custom(R)
    % Conversion de Matriz de Rotacion a Cuaternion [w, x, y, z]
    % Implementado manualmente para evitar dependencia del Robotics Toolbox
    tr = trace(R);
    if tr > 0
        S = sqrt(tr + 1.0) * 2;
        w = 0.25 * S;
        x = (R(3,2) - R(2,3)) / S;
        y = (R(1,3) - R(3,1)) / S;
        z = (R(2,1) - R(1,2)) / S;
    elseif (R(1,1) > R(2,2)) && (R(1,1) > R(3,3))
        S = sqrt(1.0 + R(1,1) - R(2,2) - R(3,3)) * 2;
        w = (R(3,2) - R(2,3)) / S;
        x = 0.25 * S;
        y = (R(1,2) + R(2,1)) / S;
        z = (R(1,3) + R(3,1)) / S;
    elseif R(2,2) > R(3,3)
        S = sqrt(1.0 + R(2,2) - R(1,1) - R(3,3)) * 2;
        w = (R(1,3) - R(3,1)) / S;
        x = (R(1,2) + R(2,1)) / S;
        y = 0.25 * S;
        z = (R(2,3) + R(3,2)) / S;
    else
        S = sqrt(1.0 + R(3,3) - R(1,1) - R(2,2)) * 2;
        w = (R(2,1) - R(1,2)) / S;
        x = (R(1,3) + R(3,1)) / S;
```

```
        y = (R(2,3) + R(3,2)) / S;
        z = 0.25 * S;
    end
    q = [w, x, y, z];
    q = q / norm(q); % Normalizar el cuaternion unitario
end
```

---

4. JUSTIFICACIÓN DEL ALGORITMO Y CONCLUSIONES

Justificación del algoritmo: Se seleccionó el Algoritmo de Enjambre de Partículas (PSO) debido a la naturaleza matemática de la cinemática de manipuladores. Al calcular la cinemática directa existen múltiples combinaciones de senos y cosenos; este paisaje de búsqueda complejo genera múltiples óptimos locales en la función de costo (posturas indeseables o singulares). PSO garantiza una robusta exploración tridimensional con el componente "social" sacando rápidamente a las partículas de configuraciones articulares erróneas.

Análisis de Precisión:

- La **convergencia** demuestra que el algoritmo logra elevar su fitness exponencialmente tras unas 50 iteraciones, estabilizándose por encima de un fitness de 0.95 (lo que indica un error milimétrico con respecto al objetivo X_d, Y_d, Z_d).
- Se incorporaron exitosamente las restricciones para los **Puntos Extra** . El uso de Cuaterniones Unitarios evitó el problema computacional de las "Singularidades de Gimbal Lock" que suelen afectar a las matrices de rotación, y gracias a la función penalizadora en el fitness $\Delta q = (q_{old} - q_{new})$, la segunda trayectoria fue calculada sin saltos bruscos en los eslabones, garantizando una **continuidad articular** viable en un entorno físico real.

5. REFERENCIAS

1. Notas tomadas en clase
2. https://en.wikipedia.org/wiki/Particle_swarm_optimization